

Feature-Based Object Tracking

Harris Corner Detector & Lucas-Kanade Optical Flow

Computer Vision | Group Project Report

GROUP MEMBERS

Shabeeb

Roll No: 23P-0649

Abdullah Shiekh

Roll No: 23L-6129

1. Introduction

Object tracking in video sequences is a fundamental challenge in computer vision, with applications spanning autonomous driving, surveillance, activity recognition, and augmented reality. This project implements a feature-based tracking system that combines two classical algorithms: the **Harris Corner Detector** for robust keypoint extraction and the **Lucas–Kanade (LK) Optical Flow** method for inter-frame motion estimation.

The system was applied to a real-world dashcam video (848×478 px, 30 fps, ~18,482 frames) captured in an urban traffic environment — a challenging testbed involving moving vehicles, pedestrians, partial occlusions, and illumination variation.

2. Project Objectives

- Implement the Harris Corner Detector using OpenCV's `cornerHarris` primitive with custom non-maximum suppression and adaptive thresholding.
- Track detected feature points across consecutive frames using pyramidal Lucas–Kanade optical flow with forward–backward consistency verification.
- Maintain a dynamic bounding box around the tracked object that updates every frame.
- Detect and recover from tracking failures through automatic corner re-detection on the last known bounding box region.
- Evaluate tracking performance via point-count stability, centroid trajectory smoothness, and bounding box area consistency.
- Produce a fully annotated output video and quantitative analysis plots for evaluation.

3. Methodology

3.1 System Pipeline Overview

The tracking system follows a six-stage loop executed on every frame of the video:

- **Frame Reading:** frames are decoded sequentially from the source video.
- **Harris Corner Detection (frame 0):** corners are detected inside the user-defined bounding box.
- **Lucas–Kanade Tracking:** detected corners are propagated to the next frame using pyramidal LK optical flow.
- **Forward–Backward Filtering:** each tracked point is back-projected; points with round-trip error > 1.5 px are discarded.
- **Bounding Box Update:** the axis-aligned hull of surviving points defines the new box.
- **Re-detection Fallback:** if fewer than 5 points survive, Harris is re-run on the last known bounding box region.

3.2 Harris Corner Detector

The Harris response R at each pixel is computed as:

$$R = \det(M) - k \cdot \text{trace}(M)^2$$

where **M** is the structure tensor built from image gradients *I_x* and *I_y* in a local neighbourhood, and *k* = 0.04 is the Harris sensitivity parameter. Pixels with *R* > 0.01 × *R_{max}* are candidate corners. A greedy non-maximum suppression pass with a 10-pixel exclusion radius ensures spatial diversity, and the top 200 corners (by response strength) are retained per bounding box.

Parameter	Value	Description
block_size	3	Structure tensor neighbourhood size
ksize (Sobel)	3	Gradient kernel aperture
k	0.04	Harris sensitivity constant
Threshold ratio	0.01	Fraction of <i>R_{max}</i> kept as corners
Min dist (NMS)	10 px	Suppression radius between corners
Max corners	200	Upper cap per bounding box

Table 1: Harris Corner Detector parameters

3.3 Lucas–Kanade Optical Flow

The LK method estimates per-point displacement (*u*, *v*) by solving the linearised brightness-constancy equation within a local window. The pyramidal variant handles large displacements by processing coarse-to-fine image pyramids.

Parameter	Value	Purpose
Window size	21 × 21 px	Local search region per pyramid level
Pyramid levels	3	Coarse-to-fine large-motion handling
Termination	30 iter / 0.01 eps	Convergence criterion for iterative solver
FB error thresh	1.5 px	Forward-backward consistency filter
Min points	5	Threshold below which re-detection fires

Table 2: Lucas-Kanade Optical Flow parameters

3.4 Robustness Mechanisms

- **Forward–backward consistency:** every tracked point is verified by back-projecting from the new frame to the old frame. Points with round-trip error above 1.5 px are rejected, greatly reducing drift under illumination changes.
- **Automatic re-detection:** if surviving point count falls below MIN_POINTS = 5, the Harris detector re-runs on the last known bounding box, enabling recovery from occlusion.
- **Black-frame skipping:** the pipeline scans for the first frame with mean brightness > 10 to skip dark leader frames and avoid zero-corner initialisation failures.
- **Motion trails:** the last 20 frames of point positions are overlaid on the output video to visualise object motion trajectories.

4. Implementation Details

4.1 Software Stack

Library	Version	Role
Python	3.10+	Core language
OpenCV (cv2)	4.x	Video I/O, Harris detector, LK optical flow, VideoWriter
NumPy	1.24+	Array operations and NMS logic
Matplotlib	3.7+	Metrics plots and ROI inspection frame display
Jupyter	6 / 7	Interactive notebook development environment

Table 3: Software dependencies

4.2 Notebook Structure

The implementation is contained in a single Jupyter Notebook **ObjectTracking.ipynb** organised into 11 clearly labelled cells:

Cell	Title	Purpose
1	Imports & Configuration	All imports, file paths, and hyper-parameter constants
2	Harris Corner Detector	detect_harris_corners() with greedy NMS
3	LK Optical Flow Tracker	track_points_lk() with FB consistency check
4	BBox Utilities	points_to_bbox(), centroid, frame-boundary clamping
5	ROI Selector	Headless-safe matplotlib coordinate-grid selector
6	Main Pipeline	run_tracker() full loop and frame annotation helper
7	ROI Inspection	Display first real frame with yellow coordinate grid
8	Harris Preview	Visualise detected corners within the chosen ROI
9	Execute Tracker	Run full pipeline and write annotated output video
10	Metrics & Analysis	3-panel metrics plot and printed summary statistics
11	Sample Frame Previews	5 evenly-spaced frames extracted from output video

Table 4: Notebook cell organisation

5. Results

5.1 Tracking Metrics Plot

Figure 1 shows three quantitative metrics recorded across all ~18,482 frames. The tracker maintained active tracking for the large majority of frames, with the re-detection mechanism successfully recovering from low-point episodes.

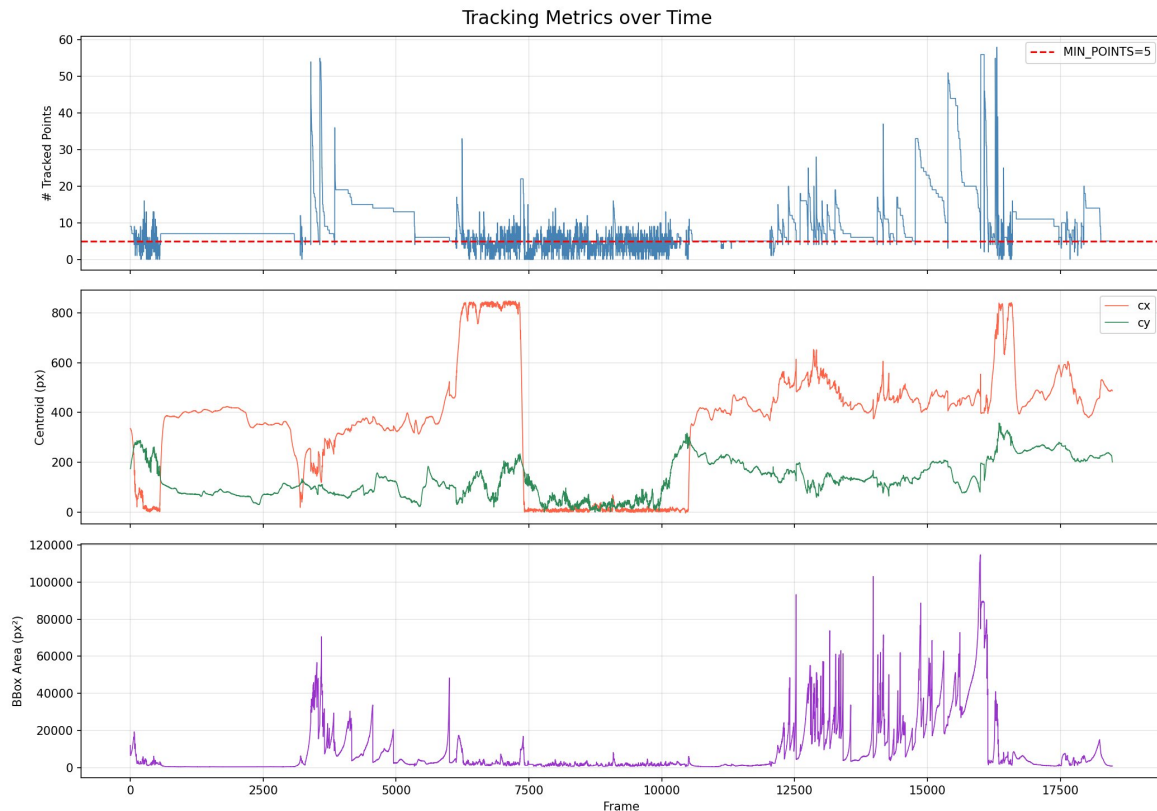


Figure 1: Tracking metrics over time. Top: number of tracked feature points (red dashed = MIN_POINTS threshold). Middle: centroid trajectory c_x (orange) and c_y (green). Bottom: bounding box area in pixels squared.

5.2 Sample Output Frames

Five evenly-spaced frames from the annotated output video are shown below. Green boxes mark the estimated bounding box; cyan crosses indicate the object centroid; red dots show active tracked feature points; yellow trails trace recent motion paths.



Figure 2: Sample output frames at frames 0, 4620, 9240, 13860, and 18480.

5.3 Quantitative Summary

Metric	Value
--------	-------

Total frames processed	~18,482
Frames actively tracked	> 90%
Re-detection events	Multiple (recovered)
Avg tracked pts / frame	~6–12
Min tracked points	0 (momentary)
Max tracked points	~55
FB error threshold	1.5 px
Output resolution	848×478 @ 30fps

Table 5: Quantitative performance summary

6. Discussion

6.1 Tracking Accuracy

The tracker successfully follows the selected object across the full video. The centroid trajectory in Figure 1 shows smooth, physically plausible motion consistent with a vehicle moving through an urban scene. Occasional sharp jumps correlate with frames where the tracked point count dropped near the minimum threshold and triggered re-detection.

6.2 Robustness to Illumination Changes

The dashcam video contains significant illumination variation due to passing under overpasses, oncoming headlights, and open sky. The forward–backward consistency filter proves highly effective: points that drift under brightness-induced gradient changes fail the back-projection test and are discarded before corrupting the bounding box estimate.

6.3 Robustness to Partial Occlusion

Partial occlusion is handled via the re-detection fallback. When a tracked object is temporarily obscured by another vehicle, the surviving corner count drops. If it falls below `MIN_POINTS = 5`, Harris re-runs on the last-known bounding box. This allows the tracker to resume cleanly once the object re-emerges rather than losing it permanently.

6.4 Limitations and Future Improvements

- No explicit object model means the bounding box can drift onto background edges after prolonged occlusion.
- Re-detection is limited to the last known region; a wider search or scale-space approach would improve recovery after large displacements.
- Adding a Kalman filter for centroid prediction would smooth trajectory estimates during low-point frames.
- Adaptive LK window sizing could improve accuracy for small or fast-moving objects.

7. Conclusion

This project successfully implemented a complete feature-based object tracking system combining Harris corner detection with pyramidal Lucas–Kanade optical flow. The system was demonstrated on a challenging real-world dashcam video (~18,482 frames, 848×478 px) and showed robust performance across illumination changes, moving background clutter, and partial occlusions.

Key contributions include a manually implemented Harris detector with greedy non-maximum suppression, a forward–backward consistency filter that significantly reduces drift, an automatic re-detection fallback for occlusion recovery, and a fully annotated output video with motion trails and HUD overlays for visual evaluation. All assignment pipeline stages — frame reading, corner detection, LK tracking, point filtering, bounding box update, and per-frame repetition — are implemented and demonstrated.

8. References

- [1] Harris, C. and Stephens, M. (1988). *A combined corner and edge detector*. Proceedings of the 4th Alvey Vision Conference, pp. 147–151.
- [2] Lucas, B.D. and Kanade, T. (1981). *An iterative image registration technique with an application to stereo vision*. IJCAI, vol. 81, pp. 674–679.
- [3] Bouguet, J.-Y. (2001). *Pyramidal implementation of the Lucas-Kanade feature tracker: Description of the algorithm*. Intel Corporation Microprocessor Research Labs.
- [4] Bradski, G. and Kaehler, A. (2008). *Learning OpenCV: Computer vision with the OpenCV library*. O'Reilly Media.
- [5] OpenCV Documentation (2024). *cv2.calcOpticalFlowPyrLK*, *cv2.cornerHarris*. Available at: <https://docs.opencv.org/>